PyIndexNum and PyIndexGUI: A Python Toolkit for Index Number Calculation

Luigi Palumbo

June 19, 2026

1 Introduction

Price and quantity index numbers are among the most widely used statistical tools in economics. National statistical offices rely on them to compute consumer price indices, producer price indices, and purchasing power parities; researchers deploy them in productivity analysis and international welfare comparisons; and central banks monitor them as core inputs to monetary policy decisions. Despite the centrality of index numbers to economic measurement, the software landscape has long been dominated by R-based solutions. Białek (Białek, 2021) introduced the `PriceIndices` package, which implements over 100 functions for bilateral and multilateral price index computation in R. Martin (Martin, 2020) contributed the `gpindex` package, offering a generalised algebraic framework for index numbers that subsumes many standard formulae as special cases. Both packages, however, require R—a language that, while powerful for statistical computing, is less prevalent in data engineering pipelines that increasingly centre on Python.

We present two complementary open-source Python libraries that fill this gap. **PyIndexNum** is a high-performance library for computing economic index numbers, built on the Polars DataFrame framework (Nahrstedt et al., 2024). It supports eight bilateral indices (Jevons, Carli, Dutot, Laspeyres, Paasche, Fisher, Törnqvist, and Walsh) and five multilateral methods (GEKS-Fisher, GEKS-Törnqvist, GEKS-Jevons, Geary–Khamis, and the Time Product Dummy), together with a suite of extension methods for chaining and splicing. **PyIndexGUI** is a desktop graphical interface, built with the Flet framework (Appveyor Systems Inc., 2024), that exposes the entire PyIndexNum pipeline through a tabbed point-and-click interface, allowing users who are unfamiliar with programming to prepare data, select methods, compute indices, and export results.

The remainder of this paper is structured as follows. Section 2 describes the technological foundations and design choices underlying both libraries. Section 3 details the data preparation pipeline. Section 4 presents the bilateral index formulae and their implementation. Section 5 covers multilateral methods and the extension methods used to maintain temporal coherence. Section 6 introduces PyIndexGUI and its interface design. Section 7 offers concluding remarks and directions for future development.

2 Architecture and Technology

2.1 Why Polars?

PyIndexNum adopts Polars rather than pandas as its data-processing backbone. Polars is a Rust-based DataFrame library with a lazy evaluation engine, multi-threaded query optimisation, and an Apache Arrow memory model (Nahrstedt et al., 2024). For the kinds of panel-dataset operations that index number computation demands—grouped aggregations, joins, pivots, and filtered selections across millions of rows—Polars delivers substantial performance gains over pandas without sacrificing readability. The library requires Python 3.10 or later and can be installed via `pip` or `uv`.

2.2 Why Flet?

PyIndexGUI is built with Flet (Appveyor Systems Inc., 2024), a Python framework that compiles to Flutter and targets Linux, macOS, Windows, and the web from a single codebase. We chose Flet over alternatives such as Streamlit or Tkinter for two reasons. First, Flet produces native desktop applications rather than browser-based dashboards, which is preferable for users who work with sensitive microdata. Second, Flet's widget set—navigation bars, data tables, dropdown selectors, and tab containers—maps naturally onto the sequential index-number workflow that PyIndexNum implements.

2.3 Design Principles

Both libraries adhere to a common set of design principles. The API is deliberately thin: each public function corresponds to a single statistical operation, and every function returns a Polars DataFrame that can be inspected, plotted, or chained into subsequent steps. Type annotations cover the entire public surface, and the test suite exercises each index formula against analytically derived results on small datasets. The libraries are licensed under the permissive MIT licence and hosted on GitHub, with documentation available on ReadTheDocs.

3 Data Preparation

Index number computation requires panel data in which individual products are observed repeatedly over time with associated prices and, for weighted indices, quantities. Real-world datasets rarely arrive in the format that formulae expect, so a reliable preparation pipeline is a prerequisite for reproducible results. PyIndexNum provides three preparation steps—standardisation, temporal aggregation, and panel balancing—each exposed as a standalone function that can be composed flexibly.

3.1 Standardisation

The `standardize_columns()` function maps user-supplied column names to the canonical schema that all downstream functions expect. A typical dataset might contain columns named `Date`, `PRDID`, `Price`, and `Qty`; standardisation renames them to `date`, `product_id`, `price`, and `quantity` while enforcing the correct Polars data types (dates, strings, and 64-bit floats). This seemingly trivial step eliminates an entire class of user errors and makes the API self-documenting.

3.2 Temporal Aggregation

Raw panel data are typically at the transaction level, but index formulae operate on period-level aggregates. The `aggregate_time()` function collapses transactions into regular intervals—daily, weekly, monthly, or quarterly—using one of several aggregation strategies. The user may select arithmetic averaging, weighted arithmetic averaging (by expenditure share), or geometric averaging for prices, and summing for quantities. This flexibility is important because the choice of aggregation method interacts with the index formula in subtle ways (International Labour Organization et al., 2004).

3.3 Panel Balancing and Imputation

Bilateral formulae compare the same set of products across exactly two periods, while multilateral methods require that every product be observed in every period within a window. Real scanner data are invariably unbalanced: products appear and disappear as assortments change. PyIndexNum offers two complementary strategies. The `remove_unbalanced()` function drops products that are absent in any period, yielding a clean but potentially much smaller panel. The `carry_forward_imputation()` function fills missing price observations with the last available price, retaining the full product set at the cost of introducing measurement noise. The appropriate strategy depends on the application, and users are encouraged to consult the CPI Manual's guidelines on treatment of missing products (International Labour Organization et al., 2004).

4 Bilateral Index Numbers

A bilateral price index compares price levels between two periods, conventionally labelled 0 (base) and 1 (current). Let p_i^t and q_i^t denote, respectively, the price and quantity of product $i = 1, \ldots, N$ in period $t \in \{0, 1\}$. Define the price relative as $r_i = p_i^1 / p_i^0$, the base-period expenditure share $s_i^0 = p_i^0 q_i^0 / \sum_j p_j^0 q_j^0$, and the current-period expenditure share $s_i^1 = p_i^1 q_i^1 / \sum_j p_j^1 q_j^1$. PyIndexNum implements eight bilateral formulae, which we group into unweighted and weighted families.

4.1 Unweighted Indices

Unweighted formulae ignore quantities entirely and base the index on price relatives alone. The three classical unweighted formulae are:

- **Jevons index** (Jevons, 1865) — the geometric mean of price relatives:

$$P_{\mathrm{J}} = \prod_{i=1}^{N} r_i^{1/N};$$

- **Carli index** (Carli, 1804) — the arithmetic mean of price relatives:

$$P_{\mathrm{C}} = \frac{1}{N} \sum_{i=1}^{N} r_i;$$

- **Dutot index** — the ratio of arithmetic mean prices:

$$P_{\mathrm{D}} = \frac{\sum_{i=1}^{N} p_i^1}{\sum_{i=1}^{N} p_i^0}.$$

The Jevons index is preferred by most statistical agencies as an elementary index because it satisfies the time-reversal test and does not exhibit the upward bias that characterises the Carli index (Levell, 2015). The Dutot index, while intuitive, is sensitive to the units in which prices are quoted and is therefore less commonly used in official statistics.

4.2 Weighted Indices

Weighted formulae incorporate quantity information through expenditure shares and span the three classical families of fixed-basket, current-basket, and superlative indices:

- **Laspeyres index** (Laspeyres, 1871) — fixed base-period basket:

$$P_{\mathrm{L}} = \frac{\sum_{i=1}^{N} p_i^1 q_i^0}{\sum_{i=1}^{N} p_i^0 q_i^0} = \sum_{i=1}^{N} s_i^0 \, r_i;$$

- **Paasche index** (Paasche, 1874) — current-period basket:

$$P_{\mathrm{P}} = \frac{\sum_{i=1}^{N} p_i^1 q_i^1}{\sum_{i=1}^{N} p_i^0 q_i^1} = \left(\sum_{i=1}^{N} \frac{s_i^1}{r_i} \right)^{-1};$$

- **Fisher ideal index** (Fisher, 1922) — geometric mean of Laspeyres and Paasche:

$$P_{\mathrm{F}} = \sqrt{P_{\mathrm{L}} \cdot P_{\mathrm{P}}};$$

- **Törnqvist index** (Törnqvist, 1936) — weighted geometric mean using average expenditure shares:

$$P_{\mathrm{T}} = \prod_{i=1}^{N} r_i^{(s_i^0 + s_i^1)/2};$$

- **Walsh index** (Walsh, 1901) — geometric mean of quantities as basket weights:

$$P_{\mathrm{W}} = \frac{\sum_{i=1}^{N} p_i^1 \sqrt{q_i^0 \, q_i^1}}{\sum_{i=1}^{N} p_i^0 \sqrt{q_i^0 \, q_i^1}}.$$

The Fisher, Törnqvist, and Walsh indices are classified as *superlative* (Diewert, 1976), meaning they approximate the true cost-of-living index to the second order under flexible functional forms. PyIndexNum computes all five formulae through a single consistent interface: the user passes the prepared DataFrame and selects the method, and the library returns a DataFrame containing the index value alongside the constituent price relatives and weights.

Listing 1: Computing a bilateral Fisher index

```
import polars as pl
import pyindexnum as pin

df = pin.standardize_columns(df, date_col="Date",
    price_col="Price", id_col="PRDID",
    quantity_col="Qty")
df_agg = pin.aggregate_time(df, freq="1mo")
df_bal = pin.remove_unbalanced(df_agg)
result = pin.fisher(df_bal)
print(result)
```

5 Multilateral Index Numbers and Extension Methods

Bilateral indices are inherently pairwise, but practical applications typically involve many time periods. Naively chaining bilateral comparisons—computing the index between period 0 and 1, then 1 and 2, and so on—introduces *chain drift*, a phenomenon whereby the chained index diverges from its direct counterpart as the number of links grows (von der Lippe, 2001). Multilateral methods avoid this problem by comparing all periods simultaneously within a window, at the cost of requiring that the comparison set remain fixed.

5.1 GEKS Indices

The GEKS method—independently proposed by Gini (Gini, 1931), Eltető and Köves (Eltetö & Köves, 1964), and Szulc (Szulc, 1964)—restores transitivity by taking the geometric mean of all bilateral comparisons between every pair of periods within a window. Given T periods in the window, the GEKS index for period t relative to base period b is:

$$P_{\mathrm{GEKS}}^{b,t} = \prod_{k=0}^{T-1} \left(P^{k,t} / P^{k,b} \right)^{1/T},$$

where $P^{k,t}$ is the bilateral index between periods k and t. PyIndexNum implements three GEKS variants that differ in the bilateral formula used as the building block: **GEKS-Fisher** (default, using the Fisher index), **GEKS-Törnqvist** (using the Törnqvist index), and **GEKS-Jevons** (using the Jevons index). The GEKS-Jevons variant is noteworthy because it requires only prices—no quantities—making it suitable for contexts where expenditure data are unavailable.

5.2 Geary–Khamis

The Geary–Khamis method (Geary, 1958; Khamis, 1972) takes a fundamentally different approach. Rather than composing bilateral indices, it solves a system of simultaneous equations that yields both a set of international (or inter-temporal) average prices $\mathbf{p}^*$ and a set of purchasing power parity factors $\mathbf{e}$. The system is:

$$p_i^* = \frac{\sum_{t=0}^{T-1} p_i^t / e^t q_i^t}{\sum_{t=0}^{T-1} q_i^t}, \qquad e^t = \frac{\sum_{i=1}^{N} p_i^t q_i^t}{\sum_{i=1}^{N} p_i^* q_i^t}.$$

PyIndexNum solves this system iteratively until convergence. The Geary–Khamis method produces additive results, a property that is valued in purchasing-power-parity work but that comes at the cost of potentially large approximation errors when product mixes differ substantially across periods.

5.3 Time Product Dummy

The Time Product Dummy (TPD) approach (Diewert, 2005; Summers, 1973) reframes the index-number problem as a regression. A logarithmic price equation is estimated:

$$\ln p_{it} = \alpha + \sum_{k=1}^{T-1} \delta_k \, d_k^{\text{time}} + \sum_{j=1}^{N-1} \gamma_j \, d_j^{\text{product}} + \varepsilon_{it},$$

where d_k^{time} and d_j^{product} are dummy variables for time periods and products, respectively. The estimated coefficients $\hat{\delta}_k$ serve directly as log price indices relative to the omitted base period. PyIndexNum supports both ordinary least squares and weighted least squares (using expenditure shares as weights) estimation. The TPD method is attractive because it naturally handles unbalanced panels—products that are absent in certain periods simply do not contribute observations for those period–product cells (de Haan et al., 2020).

5.4 Extension Methods

Multilateral indices are defined over fixed windows of periods. To extend them beyond the window, PyIndexNum implements four splicing methods that stitch overlapping windows together. These methods address the fundamental tension between the transitivity of multilateral indices and the need for time-series continuity:

Table 1: Splicing methods implemented in PyIndexNum.

Method	Description
Movement splice	The overlapping period in the new window is re-scaled so that its level matches the previous window, preserving month-to-month movements.
Window splice	The index in the new window is level-shifted so that the mean of the overlapping period matches the previous window.
Mean splice	Averages the index values from all overlapping windows at each point in time.
Fixed-base rolling window	Anchors each rolling window to a fixed base, producing a chain that is periodically rebased.

The choice among splicing methods is non-trivial and can materially affect index levels, particularly in the presence of high product turnover (de Haan & van der Grient, 2010). PyIndexNum does not prescribe a default; instead, it exposes all four methods and encourages users to compare results.

6 PyIndexGUI

While PyIndexNum provides a flexible programmatic interface, many practitioners—particularly those working in statistical agencies without extensive programming expertise—prefer graphical tools that allow interactive exploration of data and methods. PyIndexGUI addresses this need by wrapping the entire PyIndexNum pipeline in a desktop application.

6.1 Interface Design

The application is organised as five tabs that mirror the sequential workflow of index number computation:

1. **Data Import** — users load CSV or Excel files and map their columns to the canonical schema. The application previews the first rows of the dataset and validates data types before proceeding.
2. **Aggregation** — users select the target temporal frequency and aggregation method. The application displays summary statistics for the aggregated panel, including the number of unique products per period and the coverage ratio.
3. **Panel Balancing** — users choose whether to drop unbalanced products or apply carry-forward imputation. A bar chart shows the number of available versus matched products per period, making the extent of attrition immediately visible.
4. **Index Calculation** — users select one or more index formulae from a dropdown list. The application computes the requested indices and displays them in a time-series plot alongside a data table.

5. **Export** — results and the balanced panel can be exported to CSV or Excel for downstream analysis.

6.2 Technology and Distribution

PyIndexGUI is built with Flet (Appveyor Systems Inc., 2024) and can be run in interpreted mode (`flet run`) during development or compiled to a standalone binary (`flet build linux`) for distribution. The compiled application does not require a Python installation on the target machine, which simplifies deployment in institutional settings. The application communicates with PyIndexNum through its public API, meaning that any future extensions to the computational library—new index formulae, additional splicing methods, or support for spatial comparisons—become automatically available in the graphical interface.

7 Summary

We have introduced PyIndexNum and PyIndexGUI, two open-source Python libraries for economic index number calculation. PyIndexNum implements eight bilateral formulae, five multilateral methods, and four splicing extensions, built on a high-performance Polars backend that scales to the large datasets now common in scanner-data and web-scraped price collection. PyIndexGUI provides a graphical interface to the same functionality, lowering the barrier to entry for non-programmers.

Both libraries are available on GitHub under the MIT licence. The full API reference, tutorial notebooks, and installation instructions are hosted on ReadTheDocs. We welcome contributions of additional index formulae—for example, the Lloyd–Moulton index, the CCDI method, or the FEWS approach—as well as benchmarking studies that compare PyIndexNum's numerical output against established implementations such as PriceIndices (Białek, 2021) and gpindex (Martin, 2020).

References

Appveyor Systems Inc. (2024). Flet: Build realtime web, mobile and desktop apps in pure Python.

Białek, J. (2021). Priceindices – a new R package for bilateral and multilateral price index calculations. *Statistika: Statistics and Economy Journal*, *101*(2), 158–175.

Carli, G.-R. (1804). Del valore e della proporzione de' metalli monetati. *Scrittori Classici Italiani di Economia Politica*, *13*, 297–366.

de Haan, J., Hendriks, R., & Scholz, M. (2020). Price measurement using scanner data: Time-product dummy versus time dummy hedonic indexes. *Review of Income and Wealth*, *67*(2), 394–417. https://doi.org/10.1111/roiw.12468

de Haan, J., & van der Grient, H. A. (2010). Eliminating chain drift in price indexes based on scanner data. *Journal of Econometrics*, *161*(1), 36–46. https://doi.org/10.1016/j.jeconom.2010.05.004

Diewert, W. E. (2005). Weighted country product dummy variable regressions and index number formulae. *Review of Income and Wealth*, *51*(4), 561–570. https://doi.org/10.1111/j.1475-4991.2005.00168.x

Diewert, W. E. (1976). Exact and superlative index numbers. *Journal of Econometrics*, *4*(2), 115–145. https://doi.org/10.1016/0304-4076(76)90009-9

Eltetö, O., & Köves, P. (1964). On a problem of index number computation relating to international comparison. *Statisztikai Szemle*, *42*, 507–518.

Fisher, I. (1922). *The making of index numbers: A study of their varieties, tests, and reliability*. Houghton Mifflin.

Geary, R. C. (1958). A note on the comparison of exchange rates and purchasing power parities between countries. *Journal of the Royal Statistical Society. Series A (General)*, *121*(1), 97–99.

Gini, C. (1931). On the circular test of index numbers. *Metron*, *9*(1), 3–24.

International Labour Organization, International Monetary Fund, Organisation for Economic Co-operation and Development, Statistical Office of the European Communities, United Nations Economic Commission for Europe, & The World Bank. (2004). *Consumer price index manual: Theory and practice*. International Monetary Fund. https://doi.org/10.5089/9781589063327.069

Jevons, W. S. (1865). The variation of prices and the value of the currency since 1782. *Journal of the Statistical Society of London*, *28*, 584–614.

Khamis, S. (1972). A new system of index numbers for national and international purposes. *Journal of the Royal Statistical Society. Series A (General)*, *135*(1), 96–121.

Laspeyres, E. (1871). Die berechnung einer mittleren waarenpreissteigerung. *Jahrbücher für Nationalökonomie und Statistik*, *16*, 296–314.

Levell, P. (2015). Is the Carli index flawed?: Assessing the case for the new retail price index RPIJ. *Journal of the Royal Statistical Society. Series A (Statistics in Society)*, *178*(2), 303–336. https://doi.org/10.1111/rssa.12061

Martin, S. (2020). Gpindex: Generalized price and quantity indexes. *The R Journal*, *12*(1), 373–389. https://doi.org/10.32614/RJ-2020-063

Nahrstedt, F., Karmouche, M., Bargieł, K., Banijamali, P., Kumar, A. N. P., & Malavolta, I. (2024). An empirical study on the energy usage and performance of Pandas and Polars data analysis Python libraries. *Proceedings of the 10th International Conference on Energy Awareness in Software Engineering (EASE)*, 58–68. https://doi.org/10.1145/3661167.3661203

Paasche, H. (1874). Über die preisentwicklung der letzten jahre nach den hamburger börsennotirungen. *Jahrbücher für Nationalökonomie und Statistik*, *23*, 168–178.

Summers, R. (1973). International price comparisons based upon incomplete data. *Review of Income and Wealth*, *19*(1), 1–16.

Szulc, B. J. (1964). Indices for multiregional comparisons. *Przegląd Statystyczny*, *3*, 239–254.

Törnqvist, L. (1936). The bank of finland's consumption price index. *Bank of Finland Monthly Bulletin*, *10*, 1–8.

von der Lippe, P. (2001). *Chain indices: A study in price index theory*. Physica-Verlag.

Walsh, C. M. (1901). *The measurement of general exchange-value*. The Macmillan Company.